COMSATS University Islamabad, Attock Campus

Department of Computer Science

Course:

SOFTWARE QUALITY ENGINEERING

Lab Project

Café Management System

Submitted To

Ms. Munazza Maqsood

Submitted By Group Members:

Maryam Yaseen	FA24-BSE-006
Zoya Fiaz	FA24-BSE-022
Arooj Noreen	FA24-BSE-046
Rafia Rameen	FA24-BSE-051
Samrina Manahil	FA24-BSE-052

Date of Submission: May 18th, 2026

Project Report:

Cafe Management System

1. Introduction:

The Cafe Management System is a Python-based software application developed to manage the daily operations of a cafe. The system provides facilities for managing menu items, customer records, order handling, billing, feedback collection, and report generation.

This project was developed for the Software Quality Engineering lab to understand software development concepts along with software testing, bug detection, security analysis, and code quality evaluation. The system uses a menu-driven interface that allows users to interact easily with different modules of the application.

2. Objectives

- The main objectives of this project are:
- To manage cafe menu items and prices efficiently.
- To maintain customer information and records.
- To place and manage customer orders.
- To generate bills and receipts automatically.
- To collect customer feedback and ratings.
- To generate reports and statistics about cafe operations.
- To identify software bugs and vulnerabilities using SonarQube analysis.
- To improve understanding of software quality engineering concepts.

3. System Code

Management System Code

D: > BSE-4 RAFIA > SQE > SQE-LAB > cafe_management.py > ...

```python
# cafe_management.py - Cafe Management System
# GROUP MEMBERS : RAFIA RAMEEN, SAMRINA MANAHIL, MARYAM YASEEN, AROOJ NOREEN, ZOYA FIAZ
import os  # Bug: Unused import (SonarQube flags this)
import datetime

#  MENU DATA
MENU = {
    "Espresso": 300, "Double Espresso": 450, "Latte": 400,
    "Americano": 350, "Macchiato": 500, "Cappuccino": 420,
    "Matcha": 380, "Masala Chai": 200, "V60 Brew": 550, "Flat White": 480
}

DISCOUNT_RATES = {"student": 0.10, "senior": 0.15, "member": 0.20}
DELIVERY_CHARGE = 200
TAX_RATE = 0.17  # Code Smell: Magic number should be a named constant

#  DATA STORAGE (in-memory lists)
customers = []
orders = []
feedbacks = []
receipts = []
customer_id_counter = 1
order_id_counter = 1

#  MODULE 1 - MENU FUNCTIONS

def display_menu():
    unused_var = []  # Bug: Variable assigned but never used (SonarQube flags)
    print("\n========== CAFE MENU ==========")
    for i, (item, price) in enumerate(MENU.items(), 1):
        print(f"  {i}. {item:<20} Rs. {price}")
    print("================================")
```

```python

def get_price(item_name):
    return MENU.get(item_name, 0)

def add_menu_item(name, price):
    if name == None:  # Bug: Should use 'is None' (SonarQube flags == None)
        return False
    if price <= 0:
        return False
    MENU[name] = price
    return True

def remove_menu_item(name):
    if name in MENU:
        del MENU[name]
        return True
    return False

def calculate_price_with_tax(item_name):
    price = get_price(item_name)
    return round(price + price * TAX_RATE, 2)

#  MODULE 2 - CUSTOMER FUNCTIONS

ADMIN_PASSWORD = "admin123"  # Bug: Hardcoded password - Security vulnerability (SonarQube)

def add_customer(name, age, phone):
    global customer_id_counter
    customer = {
        "id": customer_id_counter,
        "name": name,
        "age": age,
        "phone": phone,
        "registered_at": datetime.datetime.now().strftime("%Y-%m-%d")
    }
    customer_id_counter += 1
    customers.append(customer)
    print(f"  Customer '{name}' added. ID: {customer['id']}")
    return customer

def search_customer(name):
    for c in customers:
        if c["name"].lower() == name.lower():
            return c
    return None

def delete_customer(customer_id):
    for i, c in enumerate(customers):
        if c["id"] == customer_id:
            removed = customers.pop(i)
            print(f"  Customer '{removed['name']}' deleted.")
            return True
    print("  Customer not found.")
    return False

def list_customers():
    if not customers:
        print("  No customers registered.")
        return
    print("\n===== REGISTERED CUSTOMERS =====")
    for c in customers:
        print(f"  ID:{c['id']} | {c['name']} | Age:{c['age']} | Phone:{c['phone']}")

def get_total_customers():
    return len(customers)

```

```python
#  MODULE 3 - ORDER FUNCTIONS

def place_order(customer_name, item, quantity, order_type, price_per_item, address=""):
    global order_id_counter
    try:
        total = price_per_item * quantity
        if order_type == "Home-Delivery":
            total += DELIVERY_CHARGE
        order = {
            "id": order_id_counter,
            "customer_name": customer_name,
            "item": item,
            "quantity": quantity,
            "order_type": order_type,
            "address": address,
            "total_price": total,
            "status": "Pending",
            "time": datetime.datetime.now().strftime("%H:%M")
        }
        order_id_counter += 1
        orders.append(order)
        print(f"  Order placed! ID: {order['id']} | Total: Rs. {total}")
        return order
    except:  # Bug: Bare except clause - catches everything (SonarQube flags)
        print("  Error placing order.")
        return None

def serve_order(order_id):
    for o in orders:
        if o["id"] == order_id:
            if o["status"] == "Pending":
                o["status"] = "Served"
                print(f"  Order #{order_id} marked as Served.")
                return True
            else:
                print(f"  Order #{order_id} is already {o['status']}.")
                return False
    print("  Order not found.")
    return False

def cancel_order(order_id):
    for o in orders:
        if o["id"] == order_id:
            if o["status"] == "Pending":
                o["status"] = "Cancelled"
                print(f"  Order #{order_id} cancelled.")
                return True
            else:
                print("  Cannot cancel a non-pending order.")
                return False
    print("  Order not found.")
    return False

def view_orders(status=None):
    filtered = [o for o in orders if status == None or o["status"] == status]  # Bug: == None
    if not filtered:
        print("  No orders found.")
        return
    print("\n===== ORDERS =====")
    for o in filtered:
        print(f"  #{o['id']} | {o['customer_name']} | {o['item']} x{o['quantity']} "
              f"| {o['order_type']} | Rs.{o['total_price']} | {o['status']}")

def get_pending_orders():
    return [o for o in orders if o["status"] == "Pending"]

def get_orders_by_customer(customer_name):
    return [o for o in orders if o["customer_name"].lower() == customer_name.lower()]

def get_total_orders():
    return len(orders)

```

```python
#  MODULE 4 - BILLING FUNCTIONS

def calculate_bill(order_list):
    return sum(o["total_price"] for o in order_list)

def apply_discount(total, discount_type):
    rate = DISCOUNT_RATES.get(discount_type, 0)
    return round(total - total * rate, 2)

def calculate_average_order_value(order_list):
    total = calculate_bill(order_list)
    return total / len(order_list)  # Bug: ZeroDivisionError if list is empty (SonarQube flags)

def generate_receipt(customer_name, order_list, discount_type=None):
    subtotal = calculate_bill(order_list)
    final_total = apply_discount(subtotal, discount_type) if discount_type else subtotal
    receipt = {"customer": customer_name, "subtotal": subtotal,
               "final_total": final_total, "discount_type": discount_type}
    receipts.append(receipt)
    print("\n========== RECEIPT ==========")
    print(f"  Customer : {customer_name}")
    for o in order_list:
        print(f"  {o['item']:<20} x{o['quantity']}  Rs. {o['total_price']}")
    print(f"  {'Subtotal':<26} Rs. {subtotal:.2f}")
    if discount_type:
        rate = DISCOUNT_RATES.get(discount_type, 0)
        print(f"  Discount ({discount_type})        {int(rate*100)}% off")
    print(f"  {'TOTAL':<26} Rs. {final_total:.2f}")
    print("==============================")
    return receipt

def get_total_earnings():
    return round(sum(r["final_total"] for r in receipts), 2)

# Code Smell: Duplicate of get_total_earnings (SonarQube flags duplicate code)
def get_total_revenue():
    return round(sum(r["final_total"] for r in receipts), 2)
    print("Revenue calculated.")  # Bug: Unreachable code after return (SonarQube flags)

#  MODULE 5 - FEEDBACK FUNCTION

def add_feedback(customer_name, rating, comment):
    if rating < 1 or rating > 5:
        print("  Rating must be between 1 and 5.")
        return False
    unused_count = 0  # Bug: Variable assigned but never used (SonarQube flags)
    fb = {"customer_name": customer_name, "rating": rating, "comment": comment}
    feedbacks.append(fb)
    print("  Thank you for your feedback!")
    return True

def view_feedback():
    if not feedbacks:
        print("  No feedback available yet.")
        return
    print("\n===== CUSTOMER FEEDBACK =====")
    for fb in feedbacks:
        stars = "★" * fb["rating"] + "☆" * (5 - fb["rating"])
        print(f"  {fb['customer_name']:<15} {stars}  \"{fb['comment']}\"")

def get_average_rating():
    if not feedbacks:
        return 0
    return round(sum(fb["rating"] for fb in feedbacks) / len(feedbacks), 2)

def get_positive_feedback():
    return [fb for fb in feedbacks if fb["rating"] >= 4]

def get_negative_feedback():
    return [fb for fb in feedbacks if fb["rating"] <= 2]
```

```
241
242 def get_feedback_count():
243     return len(feedbacks)
244
245 #  MODULE 6 - REPORTS
246
247 def show_reports():
248     print("\n========== REPORTS & STATISTICS ==========")
249     print(f"  Total Customers  : {get_total_customers()}")
250     print(f"  Total Orders     : {get_total_orders()}")
251     print(f"  Pending Orders   : {len(get_pending_orders())}")
252     print(f"  Total Earnings   : Rs. {get_total_earnings():.2f}")
253     print(f"  Average Rating   : {get_average_rating():.2f} / 5.00")
254     print(f"  Positive Reviews : {len(get_positive_feedback())}")
255     print(f"  Negative Reviews : {len(get_negative_feedback())}")
256     print("==========================================")
257
258 #  MAIN MENU DRIVER
259
260 def customer_menu():
261     while True:
262         print("\n--- Customer Management ---")
263         print("  1. Add Customer   2. Search Customer")
264         print("  3. List Customers 4. Delete Customer   0. Back")
265         ch = input("Choice: ").strip()
266         if ch == "1":
267             n = input("  Name  : "); a = int(input("  Age   : ")); p = input("  Phone : ")
268             add_customer(n, a, p)
269         elif ch == "2":
270             c = search_customer(input("  Search name: "))
271             print(f"  Found: {c}" if c else "  Not found.")
272         elif ch == "3":
273             list_customers()
274         elif ch == "4":
275             delete_customer(int(input("  Customer ID: ")))
276         elif ch == "0":
277             break
278
279 def order_menu():
280     while True:
281         print("\n--- Order Management ---")
282         print("  1.Place Order  2.View All  3.Pending  4.Serve  5.Cancel  0.Back")
283         ch = input("Choice: ").strip()
284         if ch == "1":
285             display_menu()
286             cn = input("  Customer name : "); item = input("  Item name     : ")
287             price = get_price(item)
288             if price == 0: print("  Item not in menu!"); continue
289             qty = int(input("  Quantity      : "))
290             print("  1.Take-Away  2.Dine-In  3.Home-Delivery")
291             ot = {"1": "Take-Away", "2": "Dine-In", "3": "Home-Delivery"}.get(input("  Type (1/2/3)  : "), "Dine-In")
292             addr = input("  Address       : ") if ot == "Home-Delivery" else ""
293             place_order(cn, item, qty, ot, price, addr)
294         elif ch == "2": view_orders()
295         elif ch == "3": view_orders("Pending")
296         elif ch == "4": serve_order(int(input("  Order ID: ")))
297         elif ch == "5": cancel_order(int(input("  Order ID: ")))
298         elif ch == "0": break
299
300 def billing_menu():
301     while True:
302         print("\n--- Billing ---")
303         print("  1.Generate Receipt  2.Total Earnings  0.Back")
304         ch = input("Choice: ").strip()
305         if ch == "1":
306             cn = input("  Customer name: ")
307             served = [o for o in orders if o["customer_name"].lower() == cn.lower() and o["status"] == "Served"]
308             if not served: print("  No served orders for this customer."); continue
```

```
            disc = input("  Discount (student/senior/member or Enter to skip): ").strip() or None
            generate_receipt(cn, served, disc)
        elif ch == "2":
            print(f"\n  Total Earnings: Rs. {get_total_earnings():.2f}")
        elif ch == "0":
            break

def feedback_menu():
    while True:
        print("\n--- Feedback ---")
        print("  1.Submit  2.View All  3.Average Rating  0.Back")
        ch = input("Choice: ").strip()
        if ch == "1":
            add_feedback(input("  Name: "), int(input("  Rating(1-5): ")), input("  Comment: "))
        elif ch == "2": view_feedback()
        elif ch == "3": print(f"  Average: {get_average_rating():.2f}/5")
        elif ch == "0": break

def main():
    print("\n" + "="*45)
    print("      AROOJ'S CAFE MANAGEMENT SYSTEM")
    print("    COMSATS University, Attock Campus")
    print("="*45)
    print("  Welcome to Arooj's Cafe!\n")
    while True:
        print("\n MAIN MENU ")
        print("  1. View Menu          2. Customers")
        print("  3. Orders             4. Billing")
        print("  5. Feedback           6. Reports")
        print("  0. Exit")
        ch = input("Enter choice: ").strip()
        if   ch == "1": display_menu()
        elif ch == "2": customer_menu()
        elif ch == "3": order_menu()
        elif ch == "4": billing_menu()
        elif ch == "5": feedback_menu()
        elif ch == "6": show_reports()
        elif ch == "0": print("\n  Goodbye! Visit Arooj's Cafe again!\n"); break
        else: print("  Invalid choice.")

if __name__ == "__main__":
    main()
```

4. Unit Testing Code

UNIT TESTING CODE

Unit_Testing.py > reset_state

```
# test_cafe.py - Unit Tests for Cafe Management System
import unittest
import sys
import os

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
import Cafe_Management_System as cafe

def reset_state():
    """Reset all global state before each test"""
    cafe.customers.clear()
    cafe.orders.clear()
    cafe.feedbacks.clear()
    cafe.receipts.clear()
    cafe.customer_id_counter = 1
    cafe.order_id_counter = 1

#  TEST SUITE 1 - MENU
class TestMenu(unittest.TestCase):

    def setUp(self):
        cafe.MENU["Espresso"] = 300
        cafe.MENU["Latte"] = 400
```

```
24
25      def test_get_price_valid_item(self):
26          """Known item should return correct price"""
27          self.assertEqual(cafe.get_price("Espresso"), 300)
28
29      def test_get_price_another_item(self):
30          """Latte price should be 400"""
31          self.assertEqual(cafe.get_price("Latte"), 400)
32
33      def test_get_price_unknown_item_returns_zero(self):
34          """Unknown item should return 0"""
35          self.assertEqual(cafe.get_price("Burger"), 0)
36
37      def test_add_menu_item_success(self):
38          """Adding a new item should succeed"""
39          result = cafe.add_menu_item("Green Tea", 250)
40          self.assertTrue(result)
41          self.assertEqual(cafe.get_price("Green Tea"), 250)
42
43      def test_add_menu_item_none_name_fails(self):
44          """Adding item with None name should return False"""
45          result = cafe.add_menu_item(None, 250)
46          self.assertFalse(result)
47
48      def test_add_menu_item_zero_price_fails(self):
49          """Adding item with 0 price should return False"""
50          result = cafe.add_menu_item("FreeItem", 0)
51          self.assertFalse(result)
52
53      def test_add_menu_item_negative_price_fails(self):
54          """Negative price should return False"""
55          result = cafe.add_menu_item("NegItem", -50)
56          self.assertFalse(result)
57
58      def test_remove_existing_item(self):
59          """Removing an existing item should return True"""
60          result = cafe.remove_menu_item("Espresso")
61          self.assertTrue(result)
62
63      def test_remove_nonexistent_item(self):
64          """Removing a non-existent item should return False"""
65          result = cafe.remove_menu_item("FakeItem")
66          self.assertFalse(result)
67
68      def test_calculate_price_with_tax(self):
69          """Price with tax = price * 1.17"""
70          result = cafe.calculate_price_with_tax("Espresso")
71          self.assertAlmostEqual(result, 300 * 1.17, places=1)
72
73  #  TEST SUITE 2 - CUSTOMERS
74  class TestCustomers(unittest.TestCase):
75
76      def setUp(self):
77          reset_state()
78
79      def test_add_customer_returns_dict(self):
80          """add_customer should return a dictionary"""
81          c = cafe.add_customer("Ali", 25, "03001234567")
82          self.assertIsNotNone(c)
83          self.assertEqual(c["name"], "Ali")
84
85      def test_add_customer_assigns_id(self):
86          """First customer should get ID 1"""
87          c = cafe.add_customer("Sara", 30, "03009876543")
88          self.assertEqual(c["id"], 1)
89
```

```
    def test_add_multiple_customers_unique_ids(self):
        """Each customer should have a unique ID"""
        c1 = cafe.add_customer("Ali", 25, "111")
        c2 = cafe.add_customer("Sara", 30, "222")
        self.assertNotEqual(c1["id"], c2["id"])

    def test_search_customer_found(self):
        """Search should return the correct customer"""
        cafe.add_customer("Ahmed", 22, "03001111111")
        result = cafe.search_customer("Ahmed")
        self.assertIsNotNone(result)
        self.assertEqual(result["name"], "Ahmed")

    def test_search_customer_case_insensitive(self):
        """Search should be case-insensitive"""
        cafe.add_customer("Rafia", 21, "03002222222")
        result = cafe.search_customer("rafia")
        self.assertIsNotNone(result)

    def test_search_customer_not_found(self):
        """Non-existent customer should return None"""
        result = cafe.search_customer("Ghost")
        self.assertIsNone(result)

    def test_delete_existing_customer(self):
        """Deleting existing customer should return True"""
        c = cafe.add_customer("Samrina", 23, "03003333333")
        result = cafe.delete_customer(c["id"])
        self.assertTrue(result)

    def test_delete_nonexistent_customer(self):
        """Deleting invalid ID should return False"""
        result = cafe.delete_customer(999)
        self.assertFalse(result)

    def test_get_total_customers_count(self):
        """Count should match number of added customers"""
        cafe.add_customer("A", 20, "111")
        cafe.add_customer("B", 21, "222")
        cafe.add_customer("C", 22, "333")
        self.assertEqual(cafe.get_total_customers(), 3)

    def test_delete_reduces_count(self):
        """Delete should reduce count by 1"""
        c = cafe.add_customer("X", 20, "000")
        cafe.add_customer("Y", 21, "001")
        cafe.delete_customer(c["id"])
        self.assertEqual(cafe.get_total_customers(), 1)

#  TEST SUITE 3 - ORDERS
class TestOrders(unittest.TestCase):

    def setUp(self):
        reset_state()

    def test_place_order_returns_dict(self):
        """Placing an order should return an order dict"""
        o = cafe.place_order("Ali", "Espresso", 2, "Take-Away", 300)
        self.assertIsNotNone(o)

    def test_place_order_correct_total(self):
        """Total = price_per_item * quantity"""
        o = cafe.place_order("Ali", "Espresso", 2, "Take-Away", 300)
        self.assertEqual(o["total_price"], 600)
```

```
154
155     def test_home_delivery_adds_charge(self):
156         """Home-Delivery should add Rs. 200 delivery charge"""
157         o = cafe.place_order("Sara", "Latte", 1, "Home-Delivery", 400, "Kamra")
158         self.assertEqual(o["total_price"], 600)
159
160     def test_place_order_default_status_pending(self):
161         """New order status should be Pending"""
162         o = cafe.place_order("Ahmed", "Americano", 1, "Dine-In", 350)
163         self.assertEqual(o["status"], "Pending")
164
165     def test_serve_order_changes_status(self):
166         """Serving an order should change status to Served"""
167         o = cafe.place_order("Rafia", "Cappuccino", 1, "Take-Away", 420)
168         result = cafe.serve_order(o["id"])
169         self.assertTrue(result)
170         self.assertEqual(o["status"], "Served")
171
172     def test_serve_nonexistent_order_returns_false(self):
173         """Invalid order ID should return False"""
174         self.assertFalse(cafe.serve_order(999))
175
176     def test_cancel_pending_order(self):
177         """Cancelling a pending order should succeed"""
178         o = cafe.place_order("Samrina", "Matcha", 1, "Dine-In", 380)
179         result = cafe.cancel_order(o["id"])
180         self.assertTrue(result)
181         self.assertEqual(o["status"], "Cancelled")
182
183     def test_cancel_served_order_fails(self):
184         """Cannot cancel an already served order"""
185         o = cafe.place_order("Ali", "Latte", 1, "Dine-In", 400)
186         cafe.serve_order(o["id"])
187         self.assertFalse(cafe.cancel_order(o["id"]))
188
189     def test_get_pending_orders_excludes_served(self):
190         """Pending list should not include served orders"""
191         cafe.place_order("A", "Espresso", 1, "Take-Away", 300)
192         o2 = cafe.place_order("B", "Latte", 1, "Dine-In", 400)
193         cafe.serve_order(o2["id"])
194         self.assertEqual(len(cafe.get_pending_orders()), 1)
195
196     def test_get_total_orders(self):
197         """Total should count all orders regardless of status"""
198         cafe.place_order("A", "Espresso", 1, "Take-Away", 300)
199         cafe.place_order("B", "Latte", 1, "Dine-In", 400)
200         self.assertEqual(cafe.get_total_orders(), 2)
201
202 #  TEST SUITE 4 - BILLING
203 class TestBilling(unittest.TestCase):
204
205     def setUp(self):
206         reset_state()
207
208     def test_calculate_bill_single_order(self):
209         """Bill of one order = its total_price"""
210         o = cafe.place_order("Ali", "Espresso", 2, "Take-Away", 300)
211         self.assertEqual(cafe.calculate_bill([o]), 600)
212
213     def test_calculate_bill_multiple_orders(self):
214         """Bill should sum all order totals"""
215         o1 = cafe.place_order("Ali", "Espresso", 1, "Take-Away", 300)
216         o2 = cafe.place_order("Ali", "Latte", 1, "Dine-In", 400)
217         self.assertEqual(cafe.calculate_bill([o1, o2]), 700)
218
219     def test_apply_student_discount(self):
220         """Student gets 10% off"""
221         self.assertEqual(cafe.apply_discount(1000, "student"), 900.0)
```

```
222
223     def test_apply_senior_discount(self):
224         """Senior gets 15% off"""
225         self.assertEqual(cafe.apply_discount(1000, "senior"), 850.0)
226
227     def test_apply_member_discount(self):
228         """Member gets 20% off"""
229         self.assertEqual(cafe.apply_discount(1000, "member"), 800.0)
230
231     def test_apply_unknown_discount_no_change(self):
232         """Unknown type should give 0% discount"""
233         self.assertEqual(cafe.apply_discount(1000, "vip"), 1000.0)
234
235     def test_generate_receipt_saves_record(self):
236         """Receipt should appear in receipts list"""
237         o = cafe.place_order("Sara", "Americano", 1, "Dine-In", 350)
238         cafe.generate_receipt("Sara", [o])
239         self.assertEqual(len(cafe.receipts), 1)
240
241     def test_total_earnings_sum(self):
242         """Total earnings = sum of all receipt final totals"""
243         o1 = cafe.place_order("Ali", "Espresso", 1, "Take-Away", 300)
244         o2 = cafe.place_order("Sara", "Latte", 1, "Dine-In", 400)
245         cafe.generate_receipt("Ali", [o1])
246         cafe.generate_receipt("Sara", [o2])
247         self.assertEqual(cafe.get_total_earnings(), 700)
248
249     def test_earnings_with_discount(self):
250         """Earnings should reflect discounted price"""
251         o = cafe.place_order("Ahmed", "Americano", 1, "Dine-In", 1000)
252         cafe.generate_receipt("Ahmed", [o], "student")
253         self.assertEqual(cafe.get_total_earnings(), 900.0)
254
255     def test_receipt_no_discount_full_price(self):
256         """No discount means final_total equals subtotal"""
257         o = cafe.place_order("Ali", "Espresso", 1, "Take-Away", 500)
258         r = cafe.generate_receipt("Ali", [o])
259         self.assertEqual(r["final_total"], 500)
260
261 #  TEST SUITE 5 - FEEDBACK
262 class TestFeedback(unittest.TestCase):
263
264     def setUp(self):
265         reset_state()
266
267     def test_add_valid_feedback(self):
268         """Valid rating (1-5) should be accepted"""
269         self.assertTrue(cafe.add_feedback("Ali", 5, "Excellent!"))
270
271     def test_add_minimum_rating(self):
272         """Rating of 1 is valid"""
273         self.assertTrue(cafe.add_feedback("Sara", 1, "Very bad"))
274
275     def test_add_rating_above_five_fails(self):
276         """Rating > 5 should be rejected"""
277         self.assertFalse(cafe.add_feedback("Ahmed", 6, "Out of range"))
278
279     def test_add_rating_below_one_fails(self):
280         """Rating < 1 should be rejected"""
281         self.assertFalse(cafe.add_feedback("Rafia", 0, "Zero"))
282
283     def test_feedback_count_increases(self):
284         """Valid feedbacks should increase the count"""
285         cafe.add_feedback("A", 4, "Good")
286         cafe.add_feedback("B", 3, "Okay")
287         self.assertEqual(cafe.get_feedback_count(), 2)
288
289     def test_average_rating_correct(self):
290         """Average should be (4+2)/2 = 3.0"""
291         cafe.add_feedback("Ali", 4, "Good")
```

```
292         cafe.add_feedback("Sara", 2, "Bad")
293         self.assertEqual(cafe.get_average_rating(), 3.0)
294
295     def test_average_rating_no_feedback_returns_zero(self):
296         """No feedback should return average of 0"""
297         self.assertEqual(cafe.get_average_rating(), 0)
298
299     def test_positive_feedback_filter(self):
300         """Positive = rating >= 4"""
301         cafe.add_feedback("Ali", 5, "Amazing")
302         cafe.add_feedback("Sara", 2, "Bad")
303         cafe.add_feedback("Ahmed", 4, "Good")
304         self.assertEqual(len(cafe.get_positive_feedback()), 2)
305
306     def test_negative_feedback_filter(self):
307         """Negative = rating <= 2"""
308         cafe.add_feedback("Ali", 5, "Amazing")
309         cafe.add_feedback("Sara", 1, "Terrible")
310         cafe.add_feedback("Ahmed", 2, "Poor")
311         self.assertEqual(len(cafe.get_negative_feedback()), 2)
312
313     def test_all_five_stars_average(self):
314         """All 5-star ratings should average to 5.0"""
315         cafe.add_feedback("A", 5, "Perfect")
316         cafe.add_feedback("B", 5, "Loved it")
317         self.assertEqual(cafe.get_average_rating(), 5.0)
318
319 if __name__ == "__main__":
320     unittest.main(verbosity=2)
321
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

test_place_order_correct_total (__main__.TestOrders.test_place_order_correct_total)
Total = price_per_item * quantity ...   Order placed! ID: 1 | Total: Rs. 600
ok
test_place_order_default_status_pending (__main__.TestOrders.test_place_order_default_status_pending)
New order status should be Pending ...   Order placed! ID: 1 | Total: Rs. 350
ok
test_place_order_returns_dict (__main__.TestOrders.test_place_order_returns_dict)
Placing an order should return an order dict ...   Order placed! ID: 1 | Total: Rs. 600
ok
test_serve_nonexistent_order_returns_false (__main__.TestOrders.test_serve_nonexistent_order_returns_false)
Invalid order ID should return False ...   Order not found.
ok
test_serve_order_changes_status (__main__.TestOrders.test_serve_order_changes_status)
Serving an order should change status to Served ...   Order placed! ID: 1 | Total: Rs. 420
  Order #1 marked as Served.
ok

----------------------------------------------------------------------
Ran 50 tests in 0.128s

OK
PS D:\BSE-4 RAFIA\SQE\SQE-LAB\PROJECT>
```

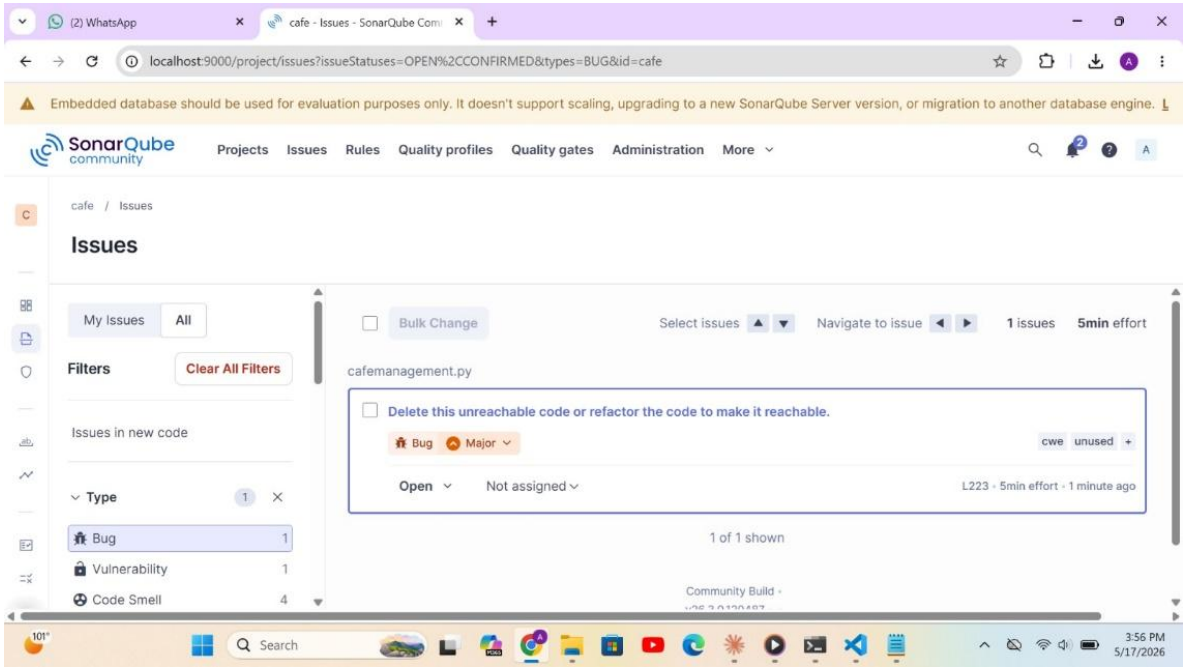
5. Bug Report:

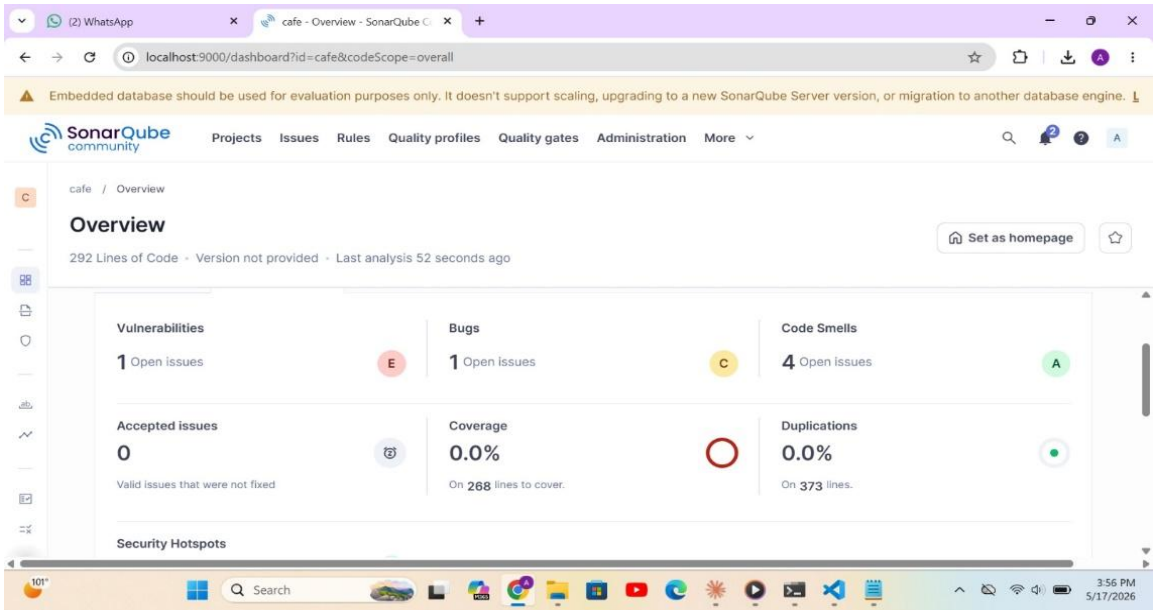
Bug Name:
Unreachable Code Detected in cafemanagement.py
Description:
SonarQube detected that certain lines of code in the cafemanagement.py file are **unreachable**, meaning the logic flow will never execute them. This may indicate leftover code, improper indentation, misplaced return/break statements, or dead code that should be removed or refactored.
Steps to Reproduce:

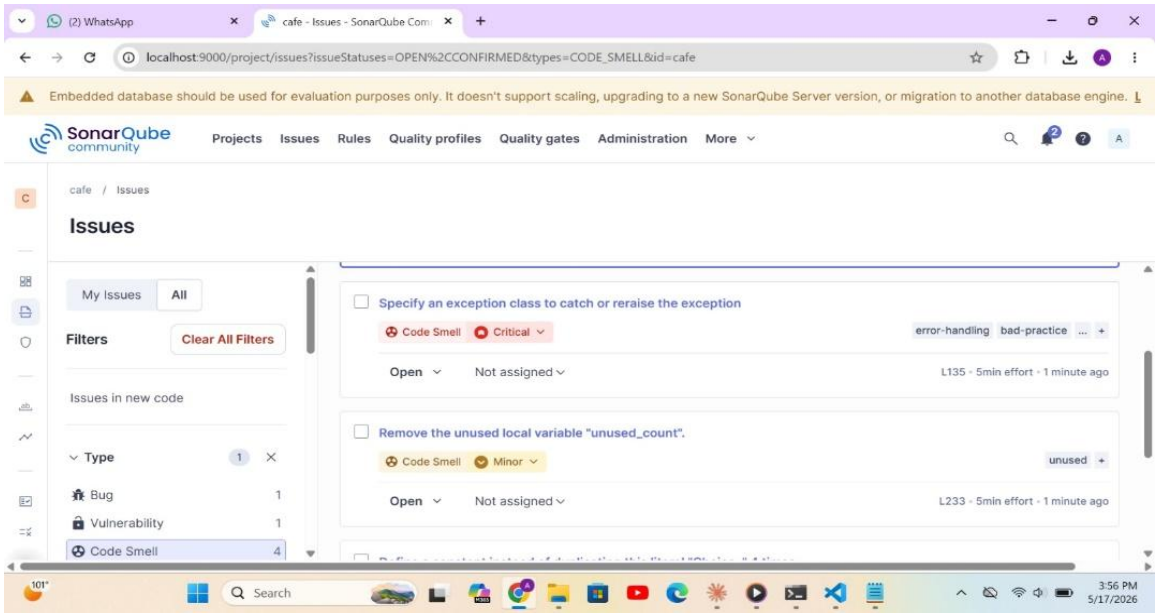
1. Open **SonarQube Dashboard**
2. Run code analysis for the project **cafe**
3. Navigate to:
   Issues → Type → Bug
4. Select the file **cafemanagement.py**
5. Observe the issue reported under line **L223**:
   "Delete this unreachable code or refactor the code to make it reachable."

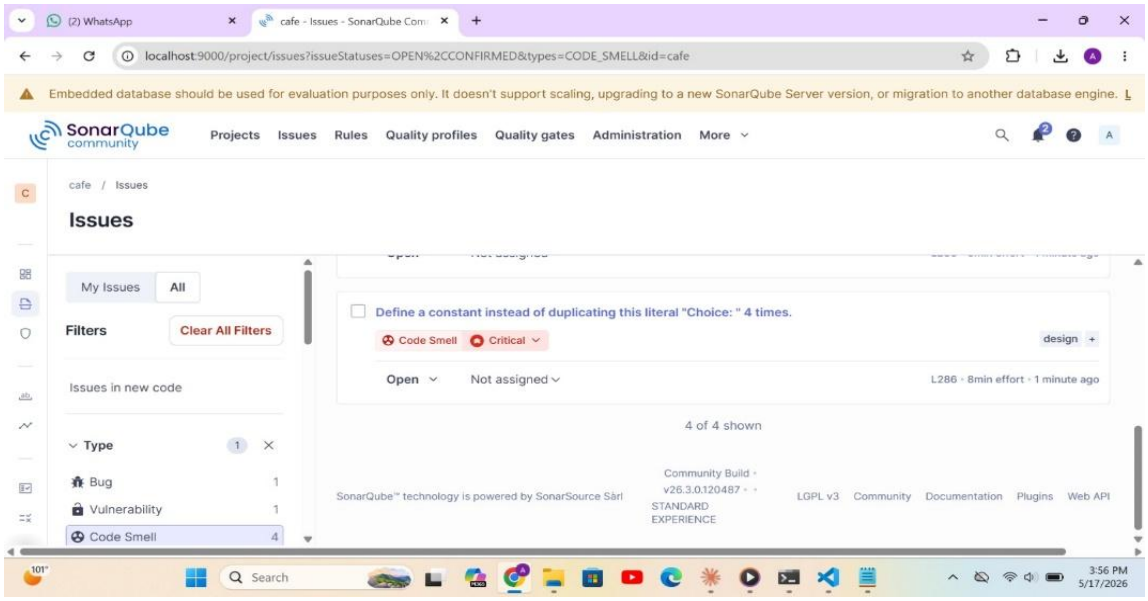
Expected Result:
The file should not contain unreachable/dead code.
All code blocks must be logically reachable through program execution.
Actual Result:
SonarQube shows a **Major Bug**, indicating unreachable code at line **223**.
This code block is not executed under any circumstance, causing redundancy and possible logical issues.
Severity:
Major
Status (Pass/Fail):
Fail (Issue still present according to SonarQube)

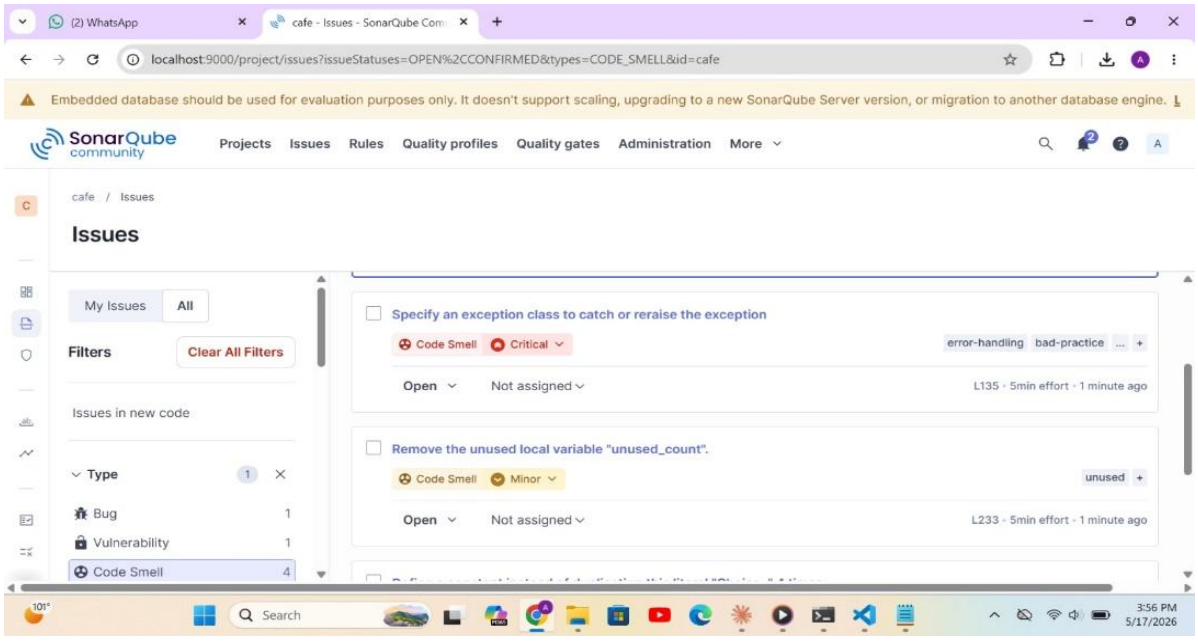
6. Sonar Cube Code Analysis:



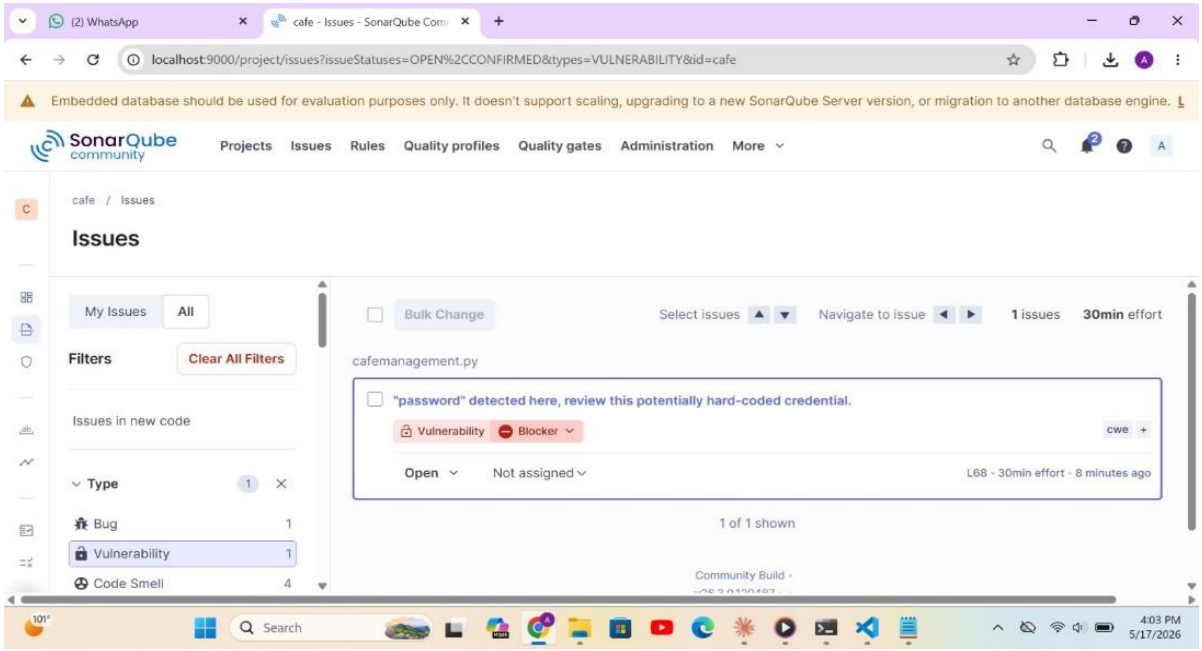

(2) WhatsApp
cafe - Overview - SonarQube
localhost:9000/dashboard?id=cafe&codeScope=overall
Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine.
SonarQube community
Projects
Issues
Rules
Quality profiles
Quality gates
Administration
More
cafe / Overview
Overview
292 Lines of Code · Version not provided · Last analysis 52 seconds ago
Set as homepage
Vulnerabilities
1 Open issues
E
Bugs
1 Open issues
C
Code Smells
4 Open issues
A
Accepted issues
0
Valid issues that were not fixed
Coverage
0.0%
On 268 lines to cover.
Duplications
0.0%
On 373 lines.
Security Hotspots
Search
3:56 PM
5/17/2026


(2) WhatsApp
cafe - Issues - SonarQube
localhost:9000/project/issues?issueStatuses=OPEN%2CCONFIRMED&types=CODE_SMELL&id=cafe
Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine.
SonarQube community
Projects
Issues
Rules
Quality profiles
Quality gates
Administration
More
cafe / Issues
Issues
My Issues
All
Filters
Clear All Filters
Issues in new code
Type
Bug
Vulnerability
Code Smell
Specify an exception class to catch or reraise the exception
Code Smell
Critical
error-handling
bad-practice
Open
Not assigned
L135 · 5min effort · 1 minute ago
Remove the unused local variable "unused_count".
Code Smell
Minor
unused
Open
Not assigned
L233 · 5min effort · 1 minute ago
Search
3:56 PM
5/17/2026


(2) WhatsApp
cafe - Issues - SonarQube
localhost:9000/project/issues?issueStatuses=OPEN%2CCONFIRMED&types=CODE_SMELL&id=cafe
Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine.
SonarQube community
Projects
Issues
Rules
Quality profiles
Quality gates
Administration
More
cafe / Issues
Issues
My Issues
All
Filters
Clear All Filters
Issues in new code
Type
Bug
Vulnerability
Code Smell
Define a constant instead of duplicating this literal "Choice: " 4 times.
Code Smell
Critical
design
Open
Not assigned
L286 · 8min effort · 1 minute ago
4 of 4 shown
SonarQube™ technology is powered by SonarSource Sàrl
Community Build
v26.3.0.120487
STANDARD EXPERIENCE
LGPL v3
Community
Documentation
Plugins
Web API
Search
3:56 PM
5/17/2026



Vulnerabilities:



7. Security Analysis:

The SonarQube scan identified two issues that impact the security and reliability of the application:

1. Improper Exception Handling (Critical)

The code uses a **generic or empty exception block**, which hides real errors and prevents proper debugging. This is a security risk because attackers can exploit hidden failures, bypass validation, or cause the system to behave unpredictably. Proper exception classes should be used, and all errors should be logged to maintain visibility and control.

2. Unused Local Variable (Minor)

An unused variable (unused_count) indicates incomplete or abandoned logic. While not a direct vulnerability, it suggests possible missing validation or security checks. It also reduces code clarity, making it harder to detect real security flaws in the future.

Overall Security Impact

- Hidden errors reduce the application's ability to detect attacks.
- Poor maintainability increases the chances of future vulnerabilities.
- Critical exception-handling issues must be fixed immediately to ensure safe and predictable behavior.

Conclusion

The Cafe Management System is a useful software application developed to automate the daily activities of a cafe. The project provides different features such as menu management, customer handling, order processing, billing, feedback collection, and report generation. It helps in reducing manual work, improving accuracy, and managing cafe operations in an organized way.

Through this project, we learned how to develop a complete Python-based management system using different modules and functions. We also gained practical knowledge about software quality engineering concepts including unit testing, bug detection, code analysis, and security evaluation using SonarQube.

The project successfully achieved its main objectives and demonstrated how software can improve efficiency and management in real-world business environments. In the future, the system can be enhanced by adding database connectivity, graphical user interfaces, and advanced security features.